

Group 42 - End-to-End Data Management Pipeline for a Recommendation System

RecoMart: E-commerce Recommendation Engine

Course	Data Management for Machine Learning - Assignment I
Submission date	22 July 2026
Team member name(s)	P. Krishna Charan, Parepalli N V L S SNEHA SREE, Vignesh K, Achhar Kanta, Gloria Thomas
BITS ID(s)	2025AB05146, 2025ab05137, 2025ab05137, 2025ab05133, 2025ab05159
Project repository	github.com/snehaparepalli14/Recommendation_Engine

1. Executive Summary

RecoMart is an e-commerce recommendation-system use case. The project objective is to improve customer engagement and cross-selling by producing personalised Top-10 product recommendations from product, user, cart, and clickstream data.

The solution uses a local data lake, validation and quarantine routing, Parquet-based preparation, a DuckDB warehouse, a versioned feature store, truncated-SVD collaborative filtering, DVC lineage tracking, and Prefect orchestration.

Project objectives

Objective	Implementation outcome
Data reliability	Collect API and CSV data with retry handling, logging, raw snapshots, checksums, and manifests
Data quality	Validate schemas, duplicates, missing values, ranges, formats, and entity references.
ML readiness	Prepare sparse interactions and engineered features for recommendation algorithms.
Reproducibility	Version data and artefacts through Git and DVC.
Operational workflow	Orchestrate the stages with Prefect and monitor task status.

2. Problem Formulation and Data Sources

The business problem is to recommend products that a customer is likely to interact with, helping RecoMart increase engagement and cross-selling opportunities. The output is a Top-10 personalised recommendation list for known users, with popularity-based fallback for unseen users.

Source	Type	Use
DummyJSON products	REST API	Product titles, category, price, rating, stock, tags, and dimensions.
DummyJSON users	REST API	User identifiers and selected non-sensitive demographic attributes.
DummyJSON carts	REST API	Cart contents, quantities, totals, and user-product relationships.
Synthetic clickstream	Local CSV	View, click, cart, and purchase-style interaction events.

3. Data Collection, Ingestion, and Raw Storage

Products, users, and carts are collected from public DummyJSON REST endpoints. A local clickstream CSV simulates interaction events. The reusable API client applies a 20-second timeout, three attempts, and exponential backoff. Each run is logged and produces a manifest.

Raw data is stored as immutable timestamped JSON or CSV snapshots partitioned by source, entity, and ingestion date. Every snapshot includes a SHA-256 checksum and record count, making later validation and reprocessing auditable.

Control	Implementation
API reliability	Timeout, retry, exponential backoff, response-status validation, and JSON parsing checks.
CSV reliability	Required-column validation before clickstream events enter raw storage.
Audit trail	Pipeline log, source metadata, run ID, timestamps, checksums, and manifest file.
Storage layout	data/raw/<source>/<entity>/ingestion_date=YYYY-MM-DD/.

4. Data Profiling and Validation

The quality gate profiles each latest raw snapshot and validates missing values, duplicates, data types, ranges, formats, and relationships. Products and users are validated first so cart and event records can be checked against trusted identifiers.

Entity	Validation outcome
Products	194 valid product records.
Users	208 valid user records; sensitive fields are not propagated to later layers.
Carts	208 valid cart records with product and user reference checks.
Events	6 valid clickstream records with valid event fields and entity references.

5. Data Preparation and Exploratory Data Analysis

Validated JSON records are flattened and standardised. Product dimensions and cart-product lists become tabular fields. Categorical fields are encoded, selected numerical fields are min-max normalised, timestamps are parsed, and sensitive user fields are excluded. Unobserved user-product pairs are treated as implicit zero-feedback instead of being stored densely.

Prepared output	Result
Products	194 cleaned product records.
Users	208 privacy-filtered user records.
Cart items	800 flattened cart-item records.
Interactions	806 user-product interactions.
Sparsity	98.0398% user-item matrix sparsity.

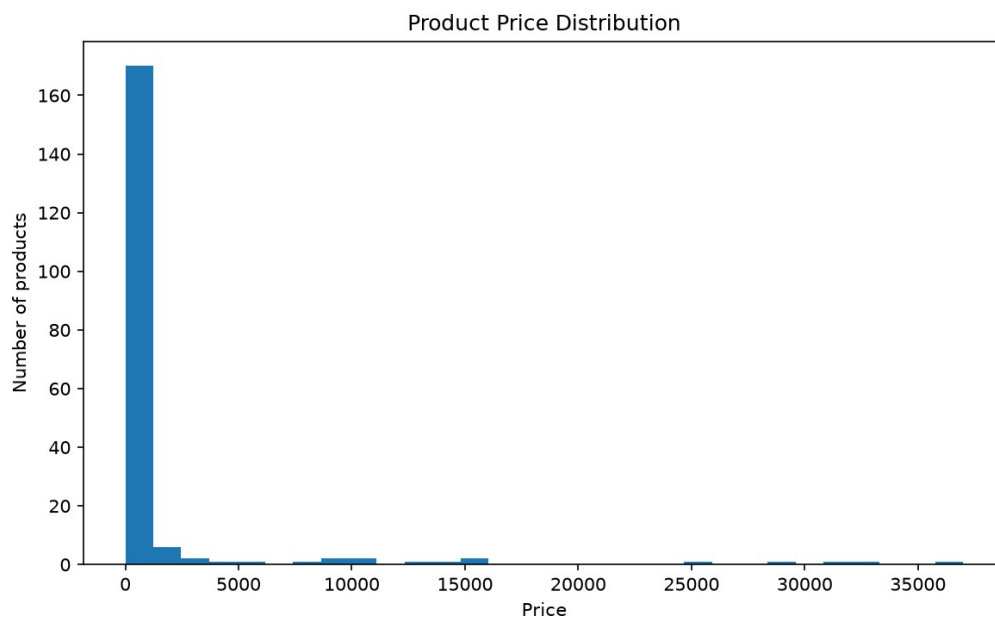


Figure 1. Product price distribution.

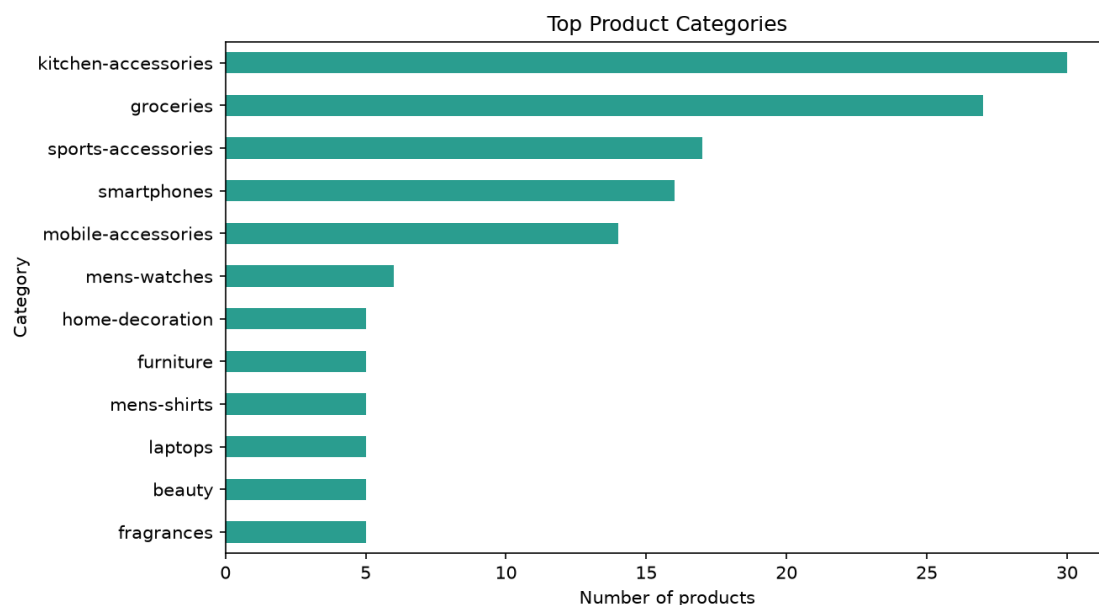


Figure 2. Product category distribution.

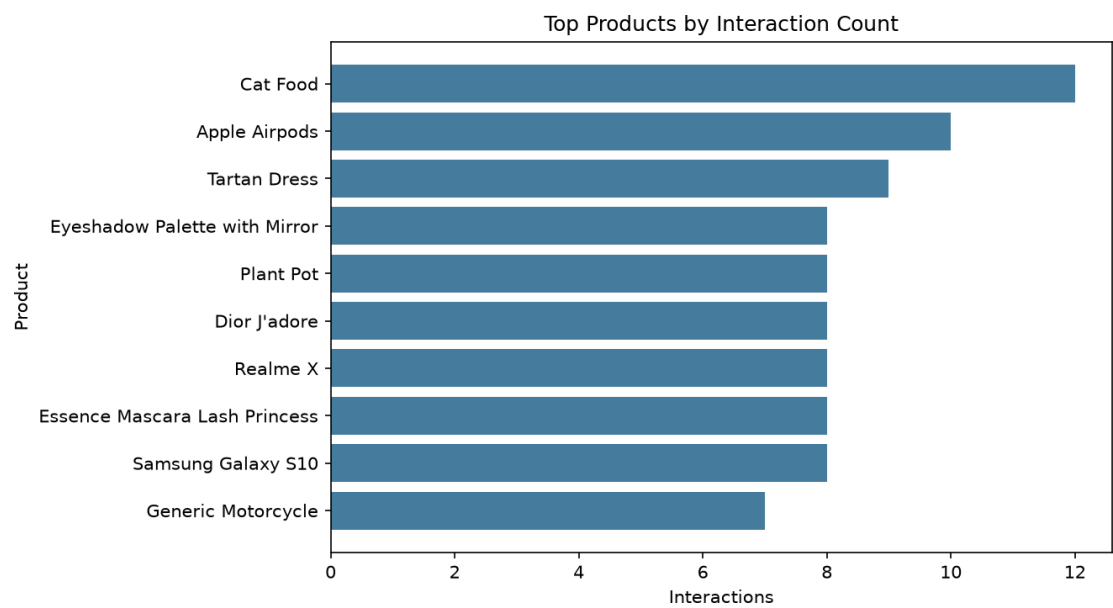


Figure 3. Top products by interaction count.

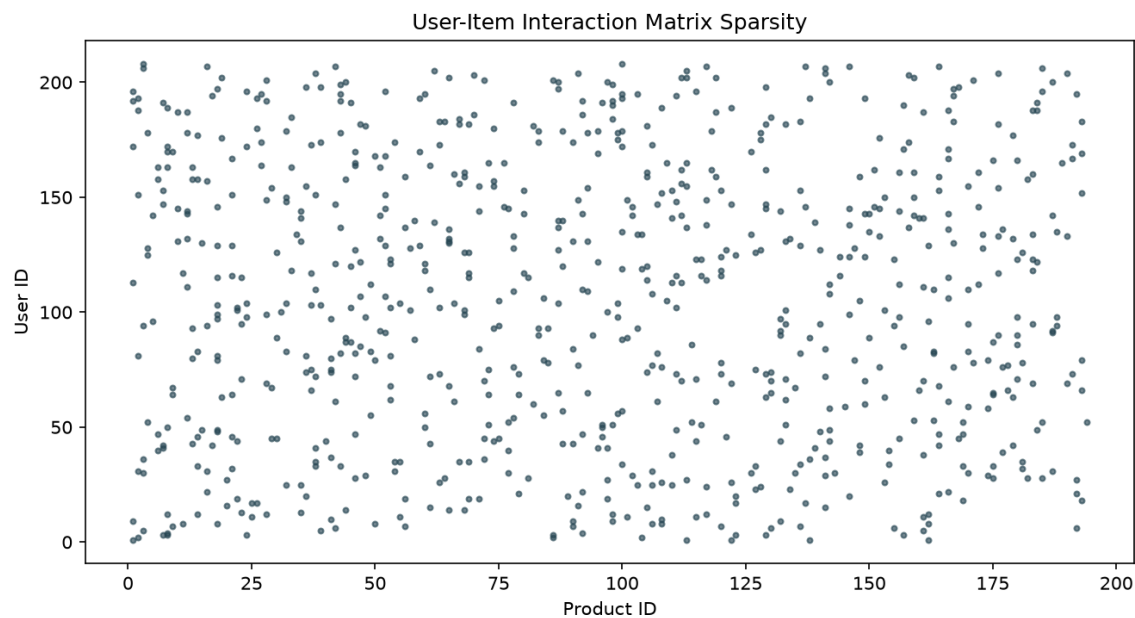


Figure 4. Sparse user-item interaction matrix.

6. Feature Engineering, Warehouse, and Feature Store

The prepared Parquet data is loaded into a portable DuckDB warehouse. The warehouse uses user and product dimensions, cart/event/interaction facts, and four feature tables. This structured layer keeps recommendation logic separate from raw operational data.

Feature view	Purpose	Rows
user_activity	Interaction count, distinct products, quantity, active days, average rating	.208
product_popularity	Interaction count, unique users, quantity, category, price, rating.	194
user_product	Weighted implicit interaction score for each observed user-product pair.	791
product_cooccurrence	Cart-based product-pair co-occurrence counts.	1,277

The custom feature store copies these feature tables into versioned DuckDB feature views. Its JSON registry records the feature name, entity keys, columns, source table, description, row count, and version. Training and inference explicitly request version v1 to avoid train-serving mismatch.

7. Data Versioning and Lineage

Git stores source code and small metadata files. DVC tracks raw, prepared, quarantine, processed, warehouse, feature-store, model, and report artefacts without committing large files directly to Git. `dvc.yaml` records the reproducible processing graph and `dvc.lock` records the dependency fingerprints used in each run.

The project was verified with DVC status reporting that data and pipelines are up to date.

8. Model Training and Evaluation

A rank-12 truncated-SVD collaborative-filtering model is trained from weighted implicit user-product interactions. Interaction scores are transformed using $\log_{1p}$. A deterministic leave-one-product-out strategy holds out one observed item per eligible user and trains on the remaining interactions.

Measure	Result
Evaluated users	208
Precision@10	0.0082
Recall@10	0.0817
NDCG@10	0.0367
Catalogue coverage	0.6443
Hit count	17

The model registry and JSON experiment tracker retain the run ID, source feature-store version, parameters, data summary, evaluation metrics, and saved model path. For known users, the inference command returns personalised recommendations; unseen users receive popularity-based fallback recommendations.

Interpretation: the modest ranking metrics are expected because the interaction matrix is sparse and the clickstream sample is small. The catalogue coverage result shows that recommendations are not concentrated on only a small set of products.

9. Pipeline Orchestration and Monitoring

Prefect orchestrates ingestion, validation, preparation/EDA, feature engineering, feature-store materialisation, and model training. Tasks have retry behaviour, logs, task status, flow status, timings, and a local JSON execution summary. The dashboard evidence shows successful flow and task runs without failures.

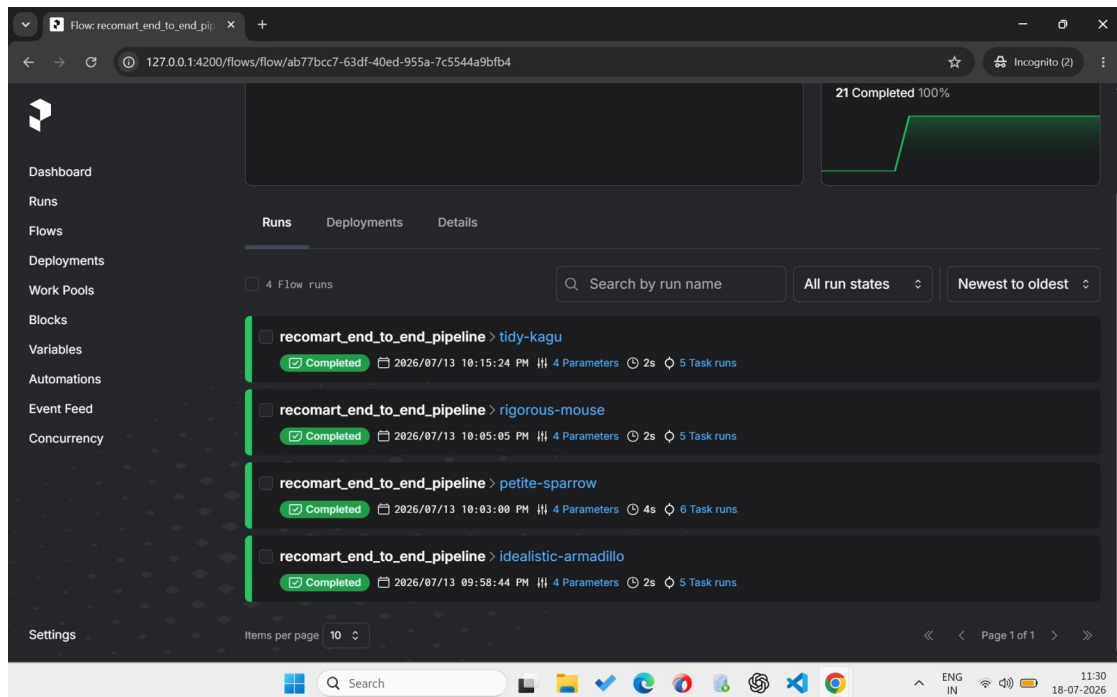


Figure 5. Prefect dashboard showing completed RecoMart flow runs and task runs.

10. DAG – Directed Acyclic Graph

RecoMart Project DAG

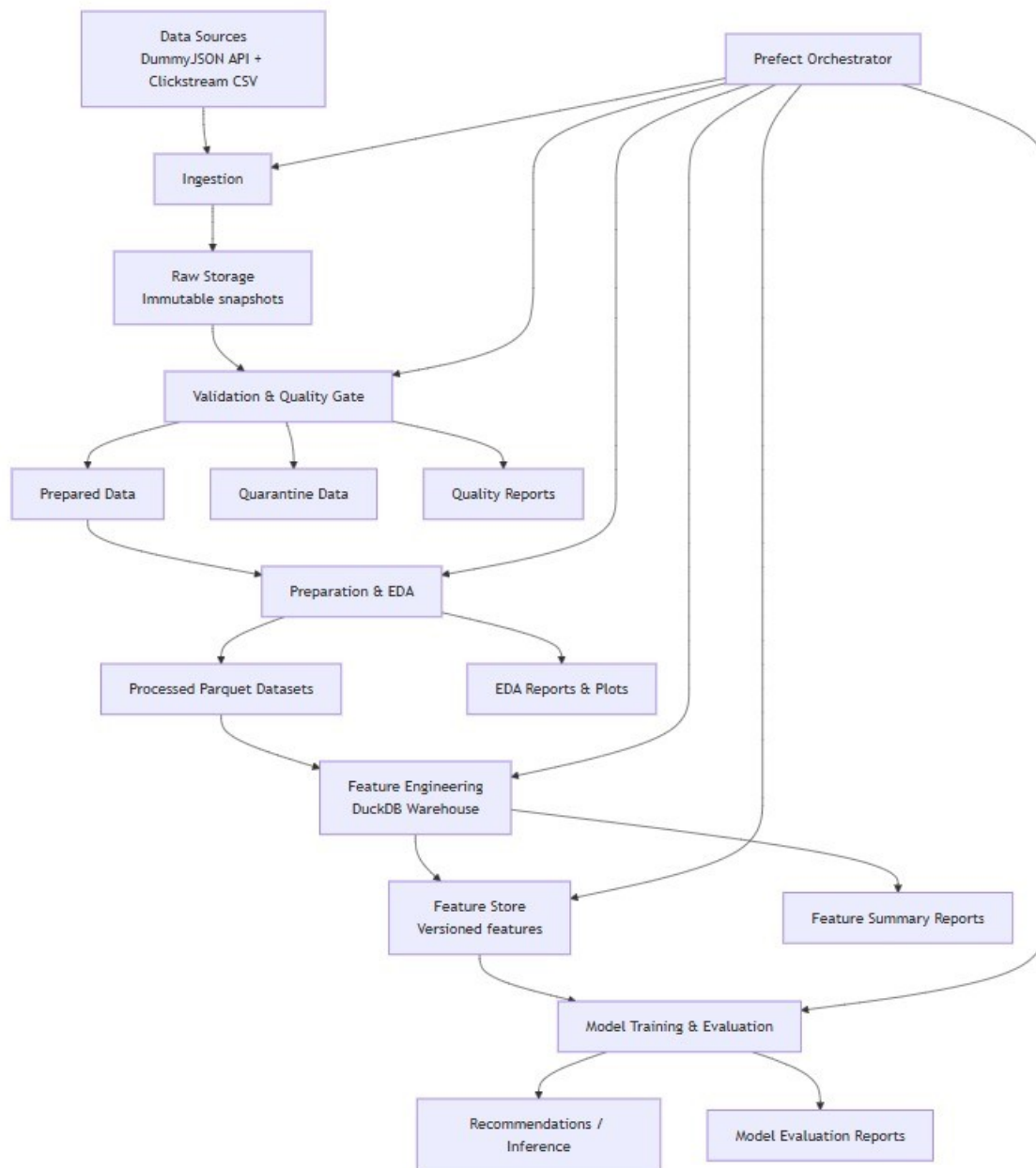


Figure 6. DAG of the RecoMart Application.

11. Testing, Results, and Reproducibility

Verification item	Result
Automated tests	19 tests passed.
Dependency consistency	pip check reported no broken requirements.
Data lineage	DVC reported data and pipelines are up to date.
Prefect monitoring	Completed runs and tasks with no failed or crashed runs.
Code repository	GitHub repository stores source, documentation, DVC metadata, and tests.

12. Conclusion and Future Scope

This project demonstrates a complete and modular data-management workflow for a recommendation system. It covers collection, storage, quality control, preparation, feature engineering, feature management, lineage, model training, evaluation, and workflow orchestration.

Future improvements include collecting richer real clickstream and purchase data, comparing SVD with content-based or hybrid recommenders, tuning hyperparameters, creating scheduled Prefect deployments, configuring remote DVC storage, and exposing recommendations through an API.

Appendix: Reproduction commands

```
python -m recomart ingest-all
python -m recomart validate-all
python -m recomart prepare-eda
python -m recomart build-features
python -m recomart materialize-feature-store
python -m recomart train-model --rank 12 --top-k 10
python -m recomart recommend --user-id 1 --limit 10
python -m recomart orchestrate-all --skip-ingestion
python -m dvc status
python -m unittest discover -s tests -v
```

Google Drive ZIP link:

[https://drive.google.com/file/d/1tsYFdEgZg243AuT_CT3lilJmtr_on-OG/view?](https://drive.google.com/file/d/1tsYFdEgZg243AuT_CT3lilJmtr_on-OG/view?usp=sharing)

[usp=sharing](https://drive.google.com/file/d/1tsYFdEgZg243AuT_CT3lilJmtr_on-OG/view?usp=sharing)

Google Drive demo-video link: